# C6 Semantic models and Direct Lake

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

## Contents

**Track C: Microsoft Fabric ..... 2**
C6 Semantic models and Direct Lake ..... 3
C6-L1 Star schemas for semantic models ..... 3
C6-L2 DAX measures that state their filter context ..... 7
C6-L3 Direct Lake, framing and the fallback you did not plan ..... 11
Track C best-practice checklist ..... 16
**Appendix: solutions and answer keys ..... 17**

# Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

# C6 Semantic models and Direct Lake

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Star schemas, DAX best practices and Power BI reports.

## C6-L1 Star schemas for semantic models

### Concept

A Power BI semantic model works best on a **star schema**: fact tables surrounded by dimension tables. Microsoft's Power BI guidance (checked September 2026) defines the parts:

- **Fact tables** store observations or events: survey responses, orders, balances. They hold dimension key columns and numeric measure columns. The dimension keys decide the fact table's **grain**, for example one row per respondent per wave per brand.
- **Dimension tables** describe business entities: respondent, brand, region, date. Each has a unique key column and descriptive columns used to filter and group. A **date dimension** belongs in almost every model.

Relationships run **one-to-many** from each dimension to the fact table, and filters flow from the dimension to the fact. Microsoft recommends that you **minimise bi-directional relationships**, which can slow queries and confuse report users with unexpected filtering. Many-to-many relationships between facts and dimensions usually mean the grain is unclear.

Build the star in the gold layer, not in the model:

- Compute derived columns such as age bands or weighted flags in Spark or SQL, in gold tables. Direct Lake on SQL analytics endpoints does not support calculated columns, and on OneLake they are in preview with restrictions. Materialised calculated columns in Import models slow refresh.
- Give every fact table a clear grain in its docstring and table comment (lesson A2-L3).
- Use surrogate or natural keys consistently, and check they are unique in each dimension (lesson C2-L2 showed warehouse keys are not enforced).

A good model is boring: one fact table per grain, conformed dimensions shared between facts, single-direction one-to-many relationships, and measures that do the arithmetic (lesson C6-L2).

## Why it matters

A brand tracker model joined a respondent table and a brand-awareness table with a bi-directional many-to-many relationship, because both held respondent_id and brand. Selecting a brand in a slicer filtered respondents, which filtered back to every brand those respondents had mentioned. Awareness for small brands showed as much higher than the survey measured. A single fact table at respondent-brand grain with one-way dimensions returned the correct figures.

## Worked example: weak code

```
1 respondents (respondent_id, age, region, weight, brand_list)
2 brand_awareness (respondent_id, brand, aware)
2 Relationship: respondents[respondent_id] <->
    brand_awareness[respondent_id], many-to-many, both directions
3 Calculated column: respondents[age_band] = SWITCH(TRUE(), ...)
```

1. **Line 1.** A list column mixes grains: one respondent, many brands.
2. **Line 3.** Bi-directional many-to-many: filters flow both ways and inflate results.
3. **Line 4.** Derived column in the model. Not supported in Direct Lake on SQL endpoints.

## Worked example: strong code

```
1 fact_awareness (respondent_key, brand_key, date_key, region_key, aware,
    weight)   grain: respondent x brand x wave
2 dim_respondent (respondent_key, age_band, gender)
3 dim_brand (brand_key, brand, category)
4 dim_date (date_key, date, wave, month)
5 dim_region (region_key, region, nation)
3 Relationships: each dim[key] 1 -> * fact_awareness[key], single direction
```

1. **Line 1.** One fact table with a stated grain.
2. **Line 2.** age_band computed in gold, not in the model.
3. **Line 6.** Filters flow one way, from dimensions to facts.

Facts with a stated grain, dimensions with unique keys, one-to-many single-direction relationships, and derived columns computed upstream.

## Recap card

- **Star schema parts** Fact tables (keys and measures, with a grain) and dimension tables (unique key, descriptive columns).
- **Relationship guidance** One-to-many, single direction. Minimise bi-directional relationships.

- **Calculated columns and Direct Lake** Not supported on SQL endpoints, preview on OneLake. Compute derived columns in gold.
- **Grain** Set by the fact table's dimension keys. State it in the table's docstring and comment.

## Practice

### Exercise 1. Multiple choice (level 1)

In a star schema, what decides the grain of a fact table?

- a) The number of measure columns
- b) The dimension keys it holds, and their values
- c) The number of rows
- d) The storage mode

**Why?**

- r1) Dimension key columns set the dimensionality, and their values set the granularity.
- r2) Grain is chosen in Power BI settings.

### Exercise 2. Multiple choice (level 2)

What does Microsoft's Power BI guidance recommend about bi-directional relationships?

- a) Use them by default so slicers work everywhere.
- b) Minimise them. They can hurt performance and confuse report users.
- c) Use them only on date tables.
- d) They are not supported in Direct Lake.

**Why?**

- r1) Bi-directional filters can slow queries and filter in ways users do not expect.
- r2) They are deprecated.

### Exercise 3. Put in order (level 2)

Order the steps to build a model for a brand tracker.

- Build the fact table with dimension keys and measure columns
- Write DAX measures
- Build dimension tables with unique keys in gold
- Create one-to-many, single-direction relationships

- State the fact table's grain

### Exercise 4. Write the function (level 3)

Write `model_warnings(relationships: list[dict[str, str]]) -> list[str]` for a model review. Each relationship has `from_table`, `to_table`, `cardinality` (`"one_to_many"` or `"many_to_many"`) and `direction` (`"single"` or `"both"`). Return, in input order, a warning for each problem:

- `"<from>-<to>: many-to-many"` for a many-to-many relationship,
- `"<from>-<to>: bi-directional"` for direction both.

A relationship with both problems produces both warnings, many-to-many first.

Your code must pass these checks:

- A one-to-many single-direction relationship has no warnings.
- Flag many-to-many and bi-directional relationships, in order.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Warnings for relationships that break star-schema guidance.
def model_warnings(
    relationships: list[dict[str, str]],
) -> list[str]: ...
```

Starter code:

```python
def model_warnings(relationships):
    return []
```

### Exercise 5. Architecture design (level 4)

A Direct Lake model on the SQL analytics endpoint needs an age band on respondents. Choose where it is built.

Options: In the gold dim_respondent table, in Spark or SQL, A calculated column in the semantic model.

1. Where age_band is computed
2. Awareness fact table grain
3. Relationship from dim_respondent to the fact

## Sources

- Fact and dimension definitions; dimension keys set dimensionality and their values granularity. (verified: https://learn.microsoft.com/en-us/power-bi/guidance/star-schema)

- Microsoft recommends minimising bi-directional relationships because of performance and confusing experiences. (verified: https://learn.microsoft.com/en-us/power-bi/guidance/relationships-bidirectional-filtering)
- Calculated columns: Direct Lake on SQL analytics endpoints no; Direct Lake on OneLake user-context only, preview. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-overview (updated 2026-09-02))

## C6-L2 DAX measures that state their filter context

### Concept

In a semantic model, arithmetic belongs in **measures**. A measure is evaluated at query time, in the **filter context** of the visual: the slicers, rows and columns that apply to that cell. A calculated column is evaluated row by row when the model refreshes and stored, which costs memory and refresh time and cannot react to slicers.

Three habits make measures correct and readable (Microsoft's DAX guidance, checked September 2026):

- **Use `DIVIDE`, not `/`.** `DIVIDE(numerator, denominator)` returns BLANK when the denominator is zero or BLANK, instead of an error or infinity. You can pass an alternate result as a third argument. Microsoft notes it is also faster than testing for zero yourself.
- **Use variables.** `VAR ... RETURN` evaluates an expression once and names it. That improves performance, readability and debugging.
- **Change filter context only with `CALCULATE`**, and say why. `CALCULATE([Weighted base], ALL(dim_brand))` removes the brand filter, which is what a share-of-total denominator needs.

A weighted share, the survey version of lesson B8-L1:

```
1 Weighted aware % =
2 VAR aware_w = SUMX ( fact_awareness, fact_awareness[aware] *
      fact_awareness[weight] )
3 VAR base_w  = SUM ( fact_awareness[weight] )
4 RETURN DIVIDE ( aware_w, base_w )
```

Put the unweighted base next to it as a separate measure, `Base (n) = DISTINCTCOUNT(fact_awareness[respondent_key])`, so report users see how many respondents stand behind each percentage.

Name measures for what they return, store them in one place (a measures table or the fact table), and set their format strings. A measure that nobody can read is a measure nobody can check.

## Why it matters

A report computed awareness as `SUM(aware_weight) / SUM(weight)` in a measure. In a new market with no fieldwork yet, the base was zero, and the visual showed "Infinity" and "NaN" in cells that the board pack exported to PDF. `DIVIDE` would have returned BLANK, and the empty cells would have been left empty.

## Worked example: weak code

```
❶ Aware % = SUM(fact_awareness[aware_weight]) / SUM(fact_awareness[weight])
❷ Brand share = [Aware %] / CALCULATE([Aware %], ALL(fact_awareness))
```

1. **Line 1.** The / operator breaks on a zero base.
2. **Line 2.** ALL on the whole fact table removes every filter, including date and region. The share compares against the wrong total.

## Worked example: strong code

```
 1 Weighted aware % =
 2 VAR aware_w = SUMX ( fact_awareness, fact_awareness[aware] *
     fact_awareness[weight] )
 3 VAR base_w = SUM ( fact_awareness[weight] )
❶ RETURN DIVIDE ( aware_w, base_w )
 5
 6 Share of aware across brands =
 7 VAR this_brand = [Weighted aware %]
❷ VAR all_brands = CALCULATE ( [Weighted aware %], ALL ( dim_brand ) )
 9 RETURN DIVIDE ( this_brand, all_brands )
```

1. **Line 4.** BLANK on a zero base, not an error.
2. **Line 8.** Removes only the brand filter. Date, region and wave filters still apply.

Measures, not calculated columns. DIVIDE for every ratio, variables for clarity, and CALCULATE with the narrowest filter change.

## Recap card

- **Measure vs calculated column** Measures compute at query time in the visual's filter context. Calculated columns are stored at refresh.
- **DIVIDE in DAX** Returns BLANK (or an alternate result) when the denominator is zero or BLANK.
- **VAR ... RETURN** Evaluates once, names parts, improves performance, readability and debugging.

- **Share of total across brands** CALCULATE([measure], ALL(dim_brand)): remove only the brand filter.

## Practice

### Exercise 1. Multiple choice (level 1)

What does `DIVIDE(10, 0)` return in DAX?

- a) An error
- b) BLANK
- c) 0
- d) Infinity

**Why?**

- r1) DIVIDE returns BLANK when the denominator is zero or BLANK, unless an alternate result is given.
- r2) DAX has no concept of division by zero.

### Exercise 2. Multiple choice (level 2)

A share-of-total measure uses `CALCULATE([Weighted aware %], ALL(fact_awareness))` as its denominator. A report filters to wave 12 and region North. What does the denominator ignore?

- a) Only the brand filter
- b) Every filter on fact_awareness, including wave and region
- c) Nothing: ALL only affects slicers
- d) Only filters from other pages

**Why?**

- r1) ALL on the fact table removes all its filters, including those that reach it through dimensions.
- r2) ALL always removes every filter in the model.

### Exercise 3. Fill the gap (level 2)

Complete the weighted share measure.

```
Weighted aware % =
_____(1) aware_w = SUMX ( fact_awareness, fact_awareness[aware] *
    fact_awareness[weight] )
VAR base_w = SUM ( fact_awareness[weight] )
RETURN _____(2) ( aware_w, base_w )
```

### Exercise 4. Write the function (level 3)

Write a measure `Share of aware across brands` that divides `[Weighted aware %]` by the same measure with the brand filter removed through `ALL ( dim_brand )`. Use variables named `this_brand` and `all_brands`, and `DIVIDE`.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
Share of aware across brands =
VAR ...
VAR ...
RETURN ...
```

Starter code:

```
Share of aware across brands = [Weighted aware %] / CALCULATE ( [Weighted
    aware %], ALL ( fact_awareness ) )
```

### Exercise 5. Architecture design (level 4)

Decide where each calculation lives in a Direct Lake model on the SQL analytics endpoint.

Options: Gold table column, DAX measure, Calculated column.

1. Respondent age band
2. Weighted awareness percentage
3. Unweighted base (number of respondents)

## Sources

- DIVIDE returns BLANK for a zero or BLANK denominator unless an alternate result is passed; it outperforms explicit zero checks. (verified: https://learn.microsoft.com/en-us/dax/best-practices/dax-divide-function-operator)
- Variables improve performance, reliability, readability and debugging. (verified: https://learn.microsoft.com/en-us/dax/best-practices/dax-variables)
- Materialised calculated columns can hurt refresh performance because values are derived at refresh time. (verified: https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-calculated-columns (updated 2026-08-23))

## C6-L3 Direct Lake, framing and the fallback you did not plan

### Concept

**Direct Lake** lets a semantic model read Delta tables in OneLake directly, loading only the columns a query needs into memory. It combines Import-like query speed with no copy of the data. A refresh does not reload data. It performs **framing**: it records which Delta table versions the model uses. Framing takes seconds, and afterwards queries see the data as of that frame. Automatic updates, a schedule or an API call can trigger framing.

There are two kinds (Microsoft Learn, updated September 2026):

- **Direct Lake on OneLake** reads Delta tables from one or more Fabric items. It never falls back to DirectQuery. If it cannot load data, the query fails.
- **Direct Lake on SQL analytics endpoints** reads one Fabric item through its endpoint. When it cannot load a table directly, it **falls back to DirectQuery**: for example when the table is a SQL view, or when the warehouse uses SQL-based granular access control. Fallback queries are slower.

The model property **Direct Lake behavior** controls fallback for the SQL kind:

- `Automatic` (default): silently fall back to DirectQuery. Reports keep working, more slowly.
- `DirectLakeOnly`: fail with an error instead. Use it in development to find fallbacks.
- `DirectQueryOnly`: always use DirectQuery, to measure the difference.

**Guardrails depend on the SKU.** For each table, Fabric limits Parquet files, row groups and rows. Over a guardrail, framing can fail (or, on the SQL kind, queries fall back). From Learn's table:

| SKU | Parquet files per table | Rows per table | Max memory |
|---|---|---|---|
| F2 to F32 | 1,000 | 300 million | 3 to 10 GB |
| F64 | 5,000 | 1.5 billion | 25 GB |
| F512 and above | 10,000 | 12 to 24 billion | 200 to 400 GB |

Small files hit the file guardrail first. Compact gold tables with `OPTIMIZE` (lesson A3-L3), and give Direct Lake tables V-Order (lesson C5-L2).

### Why it matters

A sales model on an F32 capacity was fast in testing. After six months of hourly appends, the fact table held 2,400 small Parquet files, above the F32 guardrail of 1,000. The model used Direct Lake on the SQL endpoint with the Automatic behaviour, so queries quietly fell back to DirectQuery. Reports took 40 seconds instead of two. Nobody saw an error. A nightly `OPTIMIZE` kept the table at a few dozen files and restored Direct Lake.

### Worked example: weak code

```
1 Model: sales (Direct Lake on SQL analytics endpoint), behaviour: Automatic
2 Table fact_sales: 2,400 Parquet files, appended hourly, never optimised
3 Table v_sales_clean: SQL view over fact_sales
4 Capacity: F32
```

1. **Line 1.** Automatic hides both problems: queries silently fall back.
2. **Line 2.** Above the F32 guardrail of 1,000 files per table.
3. **Line 3.** A SQL view always forces DirectQuery fallback on the SQL kind.

### Worked example: strong code

```
1 Model: sales (Direct Lake on SQL analytics endpoint), behaviour:
    DirectLakeOnly in dev, Automatic in prod
2 Table fact_sales: nightly OPTIMIZE, V-Order on, about 40 Parquet files
2 Table gold.sales_clean: Delta table written by Spark, replacing the SQL
    view
3 Capacity: F32, fact rows 120 million (limit 300 million)
5 Check: framing failures reviewed in refresh history each morning
```

1. **Line 1.** Development fails loudly on fallback. Production keeps working if one slips through.
2. **Line 3.** Materialise the view's logic as a Delta table so Direct Lake reads it directly.
3. **Line 4.** Row count checked against the SKU's guardrail.

Direct Lake stays fast when tables are Delta (not views), compact, within the SKU's guardrails, and fallback is made visible in development.

### Recap card

- **Direct Lake refresh** Framing: records Delta table versions in seconds. No data copy.
- **Direct Lake on OneLake vs on SQL** OneLake: never falls back. SQL: falls back to DirectQuery for views and SQL-based security.
- **Direct Lake behavior values** Automatic (default, silent fallback), DirectLakeOnly (error), DirectQueryOnly (always DirectQuery).

- **F64 per-table guardrails** 5,000 Parquet files, 1.5 billion rows, 25 GB max memory.

## Practice

### Exercise 1. Multiple choice (level 1)

What does a refresh of a Direct Lake semantic model do?

- a) Reloads all data into the model, like Import.
- b) Frames the model: records which Delta table versions queries will use.
- c) Rewrites the Delta tables.
- d) Switches the model to DirectQuery.

**Why?**

- r1) Framing copies metadata only, takes seconds, and sets the point in time queries see.
- r2) Direct Lake models never refresh.

### Exercise 2. Multiple choice (level 2)

A Direct Lake on SQL analytics endpoint model includes a SQL view. The Direct Lake behavior property is Automatic. What happens when a report queries the view?

- a) The query fails.
- b) The query falls back to DirectQuery and runs more slowly.
- c) The view is materialised automatically.
- d) The view is read directly from OneLake.

**Why?**

- r1) Views cannot be loaded directly from Delta, and Automatic falls back silently.
- r2) Views are always faster than tables.

### Exercise 3. Fill the gap (level 2)

Set the Direct Lake behaviour so that development fails loudly on fallback, and production keeps working.

```
Development model: Direct Lake behavior = _____(1)
Production model:  Direct Lake behavior = _____(2)
```

### Exercise 4. Write the function (level 3)

Write `guardrail_problems(sku: str, parquet_files: int, rows_millions: float) -> list[str]` using these limits from the lesson: F2 to F32 allow 1,000 files and 300 million rows per table, F64 allows 5,000 and 1,500, F128 5,000 and 3,000, F256 5,000 and 6,000. Return `"files"` and/or `"rows"` for each limit exceeded, in that order. Raise `KeyError` for other SKUs.

Your code must pass these checks:

- F2 to F32: 1,000 files and 300 million rows. Equal to the limit is allowed.
- F64: 5,000 files, 1,500 million rows. F256: 5,000 files, 6,000 million rows.
- Unknown SKUs must raise KeyError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
SMALL = ...

# Module constant.
GUARDRAILS = ...

# Direct Lake per-table guardrails a table exceeds on a SKU.
def guardrail_problems(
    sku: str,
    parquet_files: int,
    rows_millions: float,
) -> list[str]: ...
```

Starter code:

```
def guardrail_problems(sku, parquet_files, rows_millions):
    return []
```

### Exercise 5. Spot the bug (level 4)

A gold table read by Direct Lake on an F32 grows by one small file per hour and has passed the file guardrail. Tap the line that causes the build-up, and rewrite the job so the table is compacted after the append.

**What you need to know.**

`df.repartition(n)` redistributes the rows into `n` partitions. Spark writes one file per partition, so `repartition(1)` before a write produces one file per write.

`df.sparkSession` is the session the DataFrame belongs to, so a function can run SQL with `df.sparkSession.sql(...)` without a separate `spark` parameter.

```
def publish_hourly(df, table):
    """Append the latest hour to a Direct Lake gold table."""
```

```
    df.repartition(1).write.mode("append").saveAsTable(table)
    return None
```

Write down the line number of the bug. Then write the fix.

## Sources

- Direct Lake on OneLake never falls back; Direct Lake on SQL falls back to DirectQuery for SQL views and SQL-based granular access control; framing copies metadata in seconds; guardrails per SKU table. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-overview (updated 2026-09-02))
- Direct Lake behavior values Automatic (default), DirectLakeOnly and DirectQueryOnly; framing can fail when a table exceeds guardrails. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-how-it-works (updated 2025-09-22))

## Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

# Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

## C6-L1 Star schemas for semantic models

### C6-L1 exercise 1

**Hints.** (1) What does one row mean? (2) Which columns say which entities it concerns? (3) The dimension keys.

**Answer.** b) The dimension keys it holds, and their values Because: r1) Dimension key columns set the dimensionality, and their values set the granularity.

One row per combination of the dimension keys: that is the grain.

**Common wrong answers.**

- `a`: Measures are the numbers at that grain. They do not define it.
- `c`: Row count follows from the grain and the data. It does not define it.
- `d`: Import, DirectQuery or Direct Lake changes how data is read, not the grain.
- `b|r2`: Right answer, wrong reason. Grain comes from the data design.

### C6-L1 exercise 2

**Hints.** (1) Which way should filters flow in a star? (2) Dimension to fact. (3) Bi-directional breaks that.

**Answer.** b) Minimise them. They can hurt performance and confuse report users. Because: r1) Bi-directional filters can slow queries and filter in ways users do not expect.

Single-direction one-to-many relationships keep filtering predictable.

**Common wrong answers.**

- `a`: The guidance is the opposite: minimise them.
- `c`: Date tables filter facts one way, like other dimensions.
- `d`: The concern is behaviour and performance, not support.
- `b|r2`: Right answer, wrong reason. They still exist. Use them sparingly and on purpose.

### C6-L1 exercise 3

**Hints.** (1) What must be decided first? (2) Dimensions provide keys to facts. (3) Relationships before measures.

**Answer.**

1. State the fact table's grain
2. Build dimension tables with unique keys in gold
3. Build the fact table with dimension keys and measure columns
4. Create one-to-many, single-direction relationships
5. Write DAX measures

Grain, dimensions, facts, relationships, then measures.

**Common wrong answers.**

- Decide the grain before you build the fact table, or you will rebuild it.
- Measures depend on relationships to filter correctly.

### C6-L1 exercise 4

**Hints.** (1) Loop over relationships and build a name. (2) Two independent checks, not elif. (3) Append in the order given.

**Answer.**

```python
def model_warnings(relationships: list[dict[str, str]]) -> list[str]:
    """Warnings for relationships that break star-schema guidance.

    Args:
        relationships: Dicts with from_table, to_table, cardinality and direction.

    Returns:
        Warning strings in input order.
    """
    warnings = []
    for rel in relationships:
        name = f"{rel['from_table']}-{rel['to_table']}"
        if rel["cardinality"] == "many_to_many":
            warnings.append(f"{name}: many-to-many")
        if rel["direction"] == "both":
            warnings.append(f"{name}: bi-directional")
    return warnings
```

Independent checks report every problem. A review script like this runs on models exported with tools such as Tabular Editor.

### C6-L1 exercise 5

**Hints.** (1) What does Direct Lake on SQL support? (2) What does one row mean? (3) Which way should filters flow?

**Answer.**

- Compute age_band upstream: Direct Lake on SQL endpoints does not support calculated columns.
- One row per respondent, brand and wave gives a clean fact grain.
- Single-direction relationships keep filters predictable.

Do the modelling work in gold, keep the grain clean, and filter one way.

**Common wrong answers.**

- where = `calc_col`: Calculated columns are not supported in Direct Lake on SQL analytics endpoints.
- grain = `resp`: A list column mixes grains and cannot be filtered by brand.
- rel = `both`: Bi-directional filtering is rarely needed and can inflate results.

## C6-L2 DAX measures that state their filter context

### C6-L2 exercise 1

**Hints.** (1) What was DIVIDE designed for? (2) What is returned without an alternate result? (3) BLANK.

**Answer.** b) BLANK Because: r1) DIVIDE returns BLANK when the denominator is zero or BLANK, unless an alternate result is given.

BLANK keeps empty cells empty in visuals.

**Common wrong answers.**

- `a`: DIVIDE was designed to handle division by zero without an error.
- `c`: Only if you pass 0 as the alternate result.
- `d`: That is what the / operator can return. DIVIDE returns BLANK.
- `b|r2`: Right answer, wrong reason. The / operator does divide by zero. DIVIDE handles it.

### C6-L2 exercise 2

**Hints.** (1) Which table does ALL name? (2) Filters from dimensions reach the fact table. (3) Remove only the brand filter.

**Answer.** b) Every filter on fact_awareness, including wave and region Because: r1) ALL on the fact table removes all its filters, including those that reach it through dimensions.

Use `ALL(dim_brand)` to compare against all brands within the same wave and region.

**Common wrong answers.**

- `a`: That is ALL(dim_brand). ALL(fact_awareness) removes much more.
- `c`: ALL changes filter context for every filter on the table, not just slicers.
- `d`: Filter context comes from the visual and the page. ALL removes the table's filters wherever they come from.
- `b|r2`: Right answer, wrong reason. ALL removes filters from the table or column you give it.

### C6-L2 exercise 3

**Hints.** (1) The DAX keyword for a variable. (2) The safe division function. (3) `VAR` and `DIVIDE`.

**Answer.**

```
1 Weighted aware % =
2 VAR aware_w = SUMX ( fact_awareness, fact_awareness[aware] *
     fact_awareness[weight] )
3 VAR base_w = SUM ( fact_awareness[weight] )
4 RETURN DIVIDE ( aware_w, base_w )
```

Variables name each part. DIVIDE handles a zero base.

**Common wrong answers.**

- g1 = `LET`: DAX variables use VAR ... RETURN.

### C6-L2 exercise 4

**Hints.** (1) Two variables, then RETURN. (2) ALL on the brand dimension only. (3) DIVIDE the two.

**Answer.**

```
1 Share of aware across brands =
2 VAR this_brand = [Weighted aware %]
3 VAR all_brands = CALCULATE ( [Weighted aware %], ALL ( dim_brand ) )
4 RETURN DIVIDE ( this_brand, all_brands )
```

Removing only the brand filter keeps the comparison inside the same wave and region, and DIVIDE handles an empty denominator.

### C6-L2 exercise 5

**Hints.** (1) Does the value depend on the visual's filters? (2) Is it a property of one row? (3) Calculated columns are not an option here.

**Answer.**

- Age band is a property of each respondent: compute it in gold.
- Percentages depend on the visual's filters: measures.
- The base count also depends on filters: a measure, shown beside every percentage.

Row properties in gold, filter-dependent arithmetic in measures.

**Common wrong answers.**

- ageband = `calc_col`: Direct Lake on SQL analytics endpoints does not support calculated columns.
- share = `gold`: A stored percentage cannot re-aggregate correctly when users filter or group differently.
- base = `gold`: The base changes with every filter. It must be computed at query time.

## C6-L3 Direct Lake, framing and the fallback you did not plan

### C6-L3 exercise 1

**Hints.** (1) Does Direct Lake copy data? (2) What does it need to know about the tables? (3) Which version to read.

**Answer.** b) Frames the model: records which Delta table versions queries will use. Because: r1) Framing copies metadata only, takes seconds, and sets the point in time queries see.

Framing takes seconds and fixes the table versions queries see until the next frame.

**Common wrong answers.**

- `a`: Import copies data. Direct Lake refresh copies metadata only.
- `c`: The model reads Delta tables. It never writes them.
- `d`: Refresh does not change storage mode.
- `b|r2`: Right answer, wrong reason. They do refresh. The refresh is framing.

### C6-L3 exercise 2

**Hints.** (1) Can a view be read as Delta files? (2) What does Automatic do when direct loading fails? (3) Fallback.

**Answer.** b) The query falls back to DirectQuery and runs more slowly. Because: r1) Views cannot be loaded directly from Delta, and Automatic falls back silently.

Automatic keeps the report working through DirectQuery, silently. Replace views with Delta tables.

**Common wrong answers.**

- `a`: That is DirectLakeOnly, or Direct Lake on OneLake, which never falls back.
- `c`: Fabric does not materialise views for you. Write a Delta table instead.
- `d`: A view has no Delta files to read.
- `b|r2`: Right answer, wrong reason. Fallback is slower.

### C6-L3 exercise 3

**Hints.** (1) Which option errors when Direct Lake cannot be used? (2) Which is the default? (3) `DirectLakeOnly` and `Automatic`.

**Answer.**

```
Development model: Direct Lake behavior = DirectLakeOnly
Production model:  Direct Lake behavior = Automatic
```

DirectLakeOnly exposes fallback causes during development. Automatic protects users in production.

**Common wrong answers.**

- g1 = `DirectQueryOnly`: DirectQueryOnly always uses DirectQuery. It measures fallback cost but does not reveal where fallback happens.

### C6-L3 exercise 4

**Hints.** (1) A dict from SKU to (files, rows). (2) Strictly greater than the limit is a problem. (3) Index the dict so unknown SKUs raise KeyError.

**Answer.**

```
SMALL = (1000, 300)
GUARDRAILS: dict[str, tuple[int, float]] = {
    "F2": SMALL, "F4": SMALL, "F8": SMALL, "F16": SMALL, "F32": SMALL,
    "F64": (5000, 1500), "F128": (5000, 3000), "F256": (5000, 6000),
}


```

```python
def guardrail_problems(sku: str, parquet_files: int, rows_millions: float) -> list[str]:
    """Direct Lake per-table guardrails a table exceeds on a SKU.

    Raises:
        KeyError: For a SKU not in the table.
    """
    max_files, max_rows = GUARDRAILS[sku]
    problems = []
    if parquet_files > max_files:
        problems.append("files")
    if rows_millions > max_rows:
        problems.append("rows")
    return problems
```

A nightly check against the guardrails catches small-file build-up before Direct Lake falls back or framing fails.

### C6-L3 exercise 5

**Hints.** (1) How many files does each append add? (2) What merges small files? (3) OPTIMIZE after the write.

**Answer.** The bug is on line 3.

Fixed version:

```python
from pyspark.sql import DataFrame


def publish_hourly(df: DataFrame, table: str) -> None:
    """Append the latest hour, then compact small files for Direct Lake."""
    df.repartition(1).write.mode("append").saveAsTable(table)
    df.sparkSession.sql(f"OPTIMIZE {table}")
    return None
```

Hourly appends add a file each hour. OPTIMIZE compacts them, keeping the table under the guardrail. On busy tables, run it on a schedule rather than every hour.

**Common wrong answers.**

- Line 2: The docstring is fine.
- Line 4: Returning None is harmless.